WEB DEVELOPMENT NOTES

— INTRODUCTION TO WEB DEVELOPMENT —

1. What is Web Development ?

Web Development is the process of building websites and web applications that run on the internet.

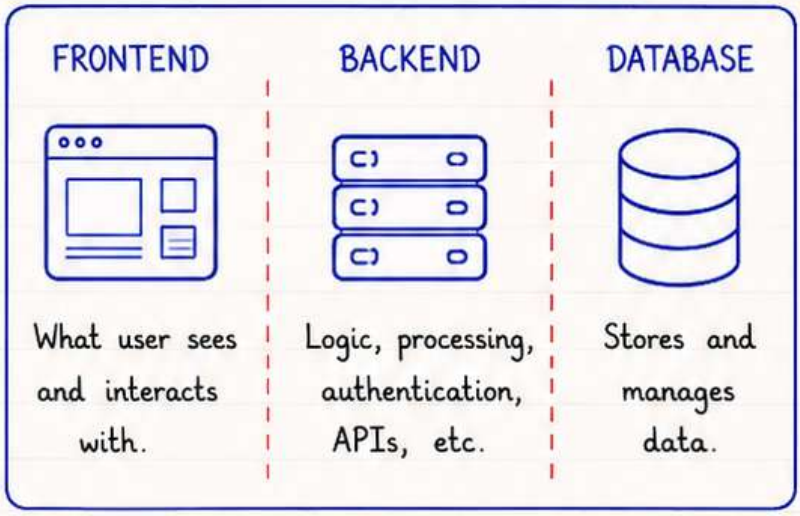
It includes:

- Designing the user interface
- Writing code
- Managing data
- Deploying and maintaining websites

In Simple Words

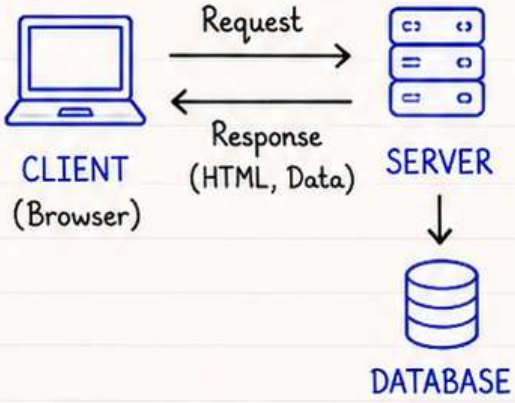
Web Development is creating websites or web apps that users can access using a browser on the internet.

2. Frontend vs Backend vs Database

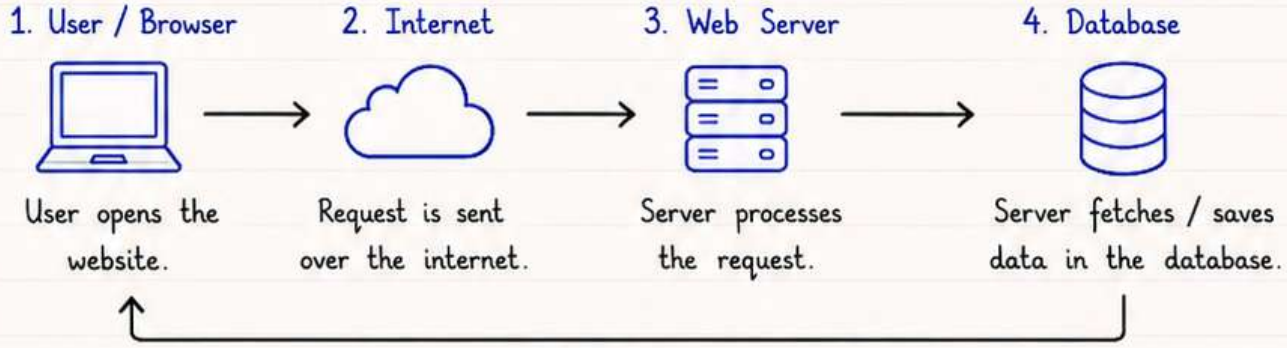


3. How a Website Works ?

A web application works on a simple request-response cycle.



4. Client → Server → Database Flow



WEB DEVELOPMENT NOTES

— HTML BASICS —

1. What is HTML ?

HTML (HyperText Markup Language) is the standard markup language used to create the structure of web pages. It tells the browser what content to display and how to organize it.

In Simple Words

HTML is the skeleton of a website. It provides the basic structure on which CSS and JavaScript add style and behavior.

2. HTML Document Structure

```
<!DOCTYPE html>
<html>
  <head>
    <title> My Page </title>
    <meta charset = "UTF-8">
  </head>
  <body>
    <!-- Page Content -->
    <h1> Hello World! </h1>
    <p> This is my first webpage. </p>
  </body>
</html>
```

- `<!DOCTYPE html>` → Declares the document type and HTML version.
- `<html>` → Root element of the HTML page.
- `<head>` → Contains meta information about the document. (not visible on the page)
- `<title>` → Title of the webpage shown on browser tab.
- `<meta>` → Provides metadata like character set, viewport, description, etc.
- `<body>` → Contains all the content that is visible on the webpage.

3. Common HTML Tags

- `<h1> to <h6>` : Headings (h1 is largest, h6 smallest)
- `<p>` : Paragraph
- `<a>` : Link
- `<img>` : Image
- `<ul>` : Unordered List
- `<ol>` : Ordered List
- `<li>` : List Item
- `<div>` : Division / Container
- `<span>` : Inline container

4. Simple HTML Page Example

```
<!DOCTYPE html>
<html>
  <head>
    <title> My First Page </title>
    <meta charset = "UTF-8">
  </head>
  <body>
    <h1> Welcome to My Website </h1>
    <p> This is a simple HTML page. </p>
    <a href = "https://www.example.com" target = "_blank">
        Visit Example Website
    </a>
  </body>
</html>
```

WEB DEVELOPMENT NOTES

— HTML FORMS —

★ 1. What is an HTML Form ?

An HTML form is used to collect user input and send it to a server for processing.

Forms are used in login pages, registration pages, search boxes, etc.

2. HTML Form Syntax

```
<form action="server-endpoint" method="GET/POST">
  <!-- form elements -->
  <input type="text" name="username">
  <button type="submit">Submit</button>
</form>
```

- action → Where data is sent.
- method → GET (show in URL)
  POST (secure, not in URL)

3. Common Form Elements

Text Input

Aman

```
<input type="text">
```

Single line text input.

Email Input

aman@example.com

```
<input type="email">
```

For email addresses.

Password Input

****

```
<input type="password">
```

Hides the characters.

Number Input

25

```
<input type="number">
```

For numbers.

Date Input

25 / 05 / 2025

```
<input type="date">
```

Select a date.

Radio Button

(•) Male
( ) Female

```
<input type="radio">
```

Select one option.

Checkbox

[x] I agree
[ ] Subscribe

```
<input type="checkbox">
```

Select multiple options.

Select (Dropdown)

India ⌄

```
<select>
  <option>India</option>
  <option>USA</option>
</select>
```

Dropdown list.

Textarea

This is a multi-line text area.

```
<textarea></textarea>
```

Multi-line text input.

Button

Submit

```
<button type="submit">Submit</button>
```

Submits the form.

GET vs POST

GET	POST
• Data is visible in URL • Limited data size • Used for search, bookmarks, etc.	• Data is not visible in URL • No size limit • Used for login, register, submitting form, etc.

4. Example : Login Form

Username : Aman
Password : •••••••
Remember me : [x]
Gender : (•) Male ( ) Female

[Login] [Reset]

```
<form action="/login" method="POST">
  <label>Username :</label>
  <input type="text" name="username"><br><br>
  <label>Password :</label>
  <input type="password" name="password"><br><br>
  <label>Remember me :</label>
  <input type="checkbox" name="remember"><br><br>
  <label>Gender :</label>
  <input type="radio" name="gender" value="male"> Male
  <input type="radio" name="gender" value="female"> Female<br><br>
  <button type="submit">Login</button>
  <button type="reset">Reset</button>
</form>
```

WEB DEVELOPMENT NOTES

– CSS BASICS –

1. What is CSS ?

CSS (Cascading Style Sheets) is used to style and format the layout of web pages. It controls colors, fonts, spacing, positioning and the overall visual appearance.

In Simple Words

CSS is the language used to make HTML elements look beautiful and presentable.

2. Ways to Add CSS

1. Inline CSS

Used inside an HTML element using the style attribute.

```
<p style="color: blue;">
  This is a paragraph.
</p>
```

2. Internal CSS

Used inside <style> tag in the <head> section.

```
<style>
  p { color: blue; }
</style>
```

3. External CSS

Used in a separate .css file. Linked using <link> tag.

```
<link rel="stylesheet"
  href="style.css">
```

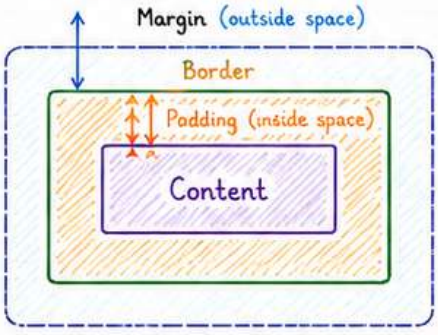
3. CSS Selectors

- Element Selector : p { } ⟶ Selects all <p> elements
- ID Selector : #id { } ⟶ Selects element with id="id"
- Class Selector : .class { } ⟶ Selects all elements with class="class"
- Universal Selector : * { } ⟶ Selects all elements on the page

Example

```
<p>Hello</p>
<p id="para">Hello</p>
<p class="text">Hello</p>
<p class="text">Bye</p>
```

4. CSS Box Model



- Content : The actual content (text, image, etc.)
- Padding : Space between content and border
- Border : Surrounds the padding and content
- Margin : Space outside the border (separates from other elements)

Box Model Order

Margin ↓ Border ↓ Padding ↓ Content

5. Basic CSS Syntax

```
selector {
    property: value;
}
```

- selector ⟶ Selector — Element you want to style
- property ⟶ Property — What you want to change
- value ⟶ Value — How you want to change it

Example

```
p {
    color: blue;
    font-size: 18px;
}
```

WEB DEVELOPMENT NOTES

— CSS LAYOUT —

1. Display

The display property defines how an element is displayed on the page.

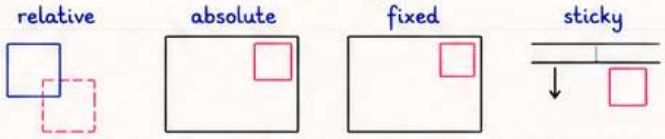
block	inline	inline-block
Takes full width. Starts on new line.	Takes only necessary width. Does not start on new line.	Takes necessary width and can set width/height.

2. Position

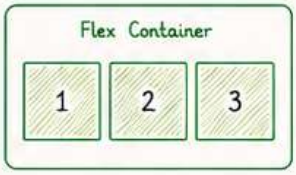
The position property specifies the type of positioning method used for an element.

- `static` – Default position (normal flow)
- `relative` – Positioned relative to its normal position
- `absolute` – Positioned relative to nearest positioned ancestor (removed from normal flow)
- `fixed` – Positioned relative to the viewport, stays in place on scroll
- `sticky` – Behaves like relative until a threshold is reached, then sticks

Position Example



3. Flexbox (One Dimension Layout)



- Used for layout in one direction (row or column).
- Parent → `display: flex;`
- Aligns and distributes space among items.

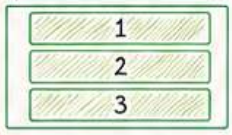
Common Properties

- `flex-direction` : row | column
- `justify-content` : aligns items horizontally
- `align-items` : aligns items vertically
- `gap` : space between items

Flex Direction Example

`flex-direction: row;`

`flex-direction: column;`



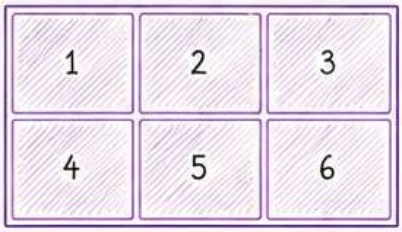
4. CSS Grid (Two Dimension Layout)

Used for layout in rows and columns.

Parent → `display: grid;`

Powerful for complex layouts.

Grid Example (3 Columns)



Common Properties

- `grid-template-columns`
- `grid-template-rows`
- `gap`
- `justify-items`
- `align-items`
- `place-items`

5. Responsive Design

Responsive design makes websites look good on all devices.

Media Queries allow you to apply CSS based on screen size.

Media Query Syntax

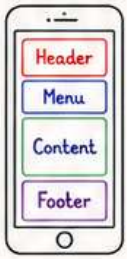
```
@media (max-width: 768px) {
  /* CSS rules */
}
```

Common Breakpoints

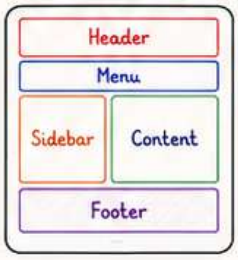
- Mobile : `max-width: 576px`
- Tablet : `max-width: 768px`
- Laptop : `max-width: 1024px`
- Desktop : `min-width: 1025px`

Example Layout on Different Screens

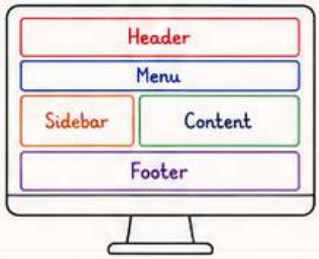
Mobile (< 576px)



Tablet (576px – 1024px)



Desktop (> 1024px)



6. Useful Units

- `px` – Fixed pixels
- `%` – Relative to parent
- `em` – Relative to font-size of parent
- `rem` – Relative to font-size of root (html)
- `vw` – 1% of viewport width
- `vh` – 1% of viewport height

Example

```
width: 50%;
font-size: 1.2rem;
height: 100vh;
```

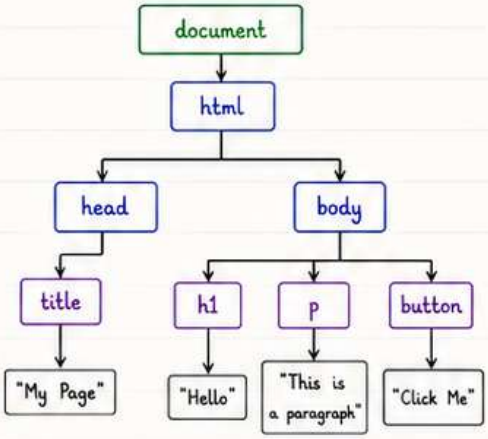
WEB DEVELOPMENT NOTES

— DOM MANIPULATION —

1. What is DOM ?

DOM (Document Object Model) is a programming interface for web documents. It represents the HTML document as a tree structure so that JavaScript can access and update the content, structure and style of the page.

2. DOM Tree Structure



Example HTML

```
<html>
  <head>
    <title>My Page</title>
</head>
<body>
  <h1>Hello</h1>
  <p>This is a paragraph</p>
  <button>Click Me</button>
</body>
</html>
```

3. Selecting Elements

- getElementById()

```
<h1 id="title">Hello</h1>
let el = document.getElementById("title");
// returns the element with id="title"
```

- getElementsByClassName()

```
<p class="text">A</p>
<p class="text">B</p>
let els = document.getElementsByClassName("text");
// returns HTMLCollection of elements
```

- querySelector()

```
<p class="text">Hello</p>
let el = document.querySelector(".text");
// returns first matched element
```

- querySelectorAll()

```
<p class="text">A</p>
<p class="text">B</p>
let els = document.querySelectorAll(".text");
// returns NodeList of all matched elements
```

Note:

- getElementById → single element
- getElementsByClassName → all elements (HTMLCollection)
- querySelector → first matched element
- querySelectorAll → all matched elements (NodeList)

4. DOM Manipulation Examples

- Change Text

```
<h1 id="title">Hello</h1>
let el = document.getElementById("title");
el.innerText = "Hi There!";
```

- Change HTML

```
<p id="para">Old</p>
let el = document.getElementById("para");
el.innerHTML = "<b>New</b>";
```

- Change Style

```
<p id="para">Text</p>
let el = document.getElementById("para");
el.style.color = "blue";
el.style.fontSize = "20px";
```

- Add Element

```
let newDiv = document.createElement("div");
newDiv.innerText = "I am new!";
document.body.appendChild(newDiv);
```

New Div → <body> ... </body>

- Remove Element

```
let el = document.getElementById("para");
el.remove();
```

 Text →

5. Events (Example)

```
<button id="btn">Click Me</button>
let btn = document.getElementById("btn");
btn.addEventListener("click", function(){
    alert("Button Clicked!");
});
```

Click Me → Alert: Button Clicked! [OK]

6. DOM Manipulation Flow

Select Element → Access / Read Element → Modify Element → Page Updates

- Select Element: getElementById() querySelector() etc.
- Access / Read Element: innerText, value, style, attributes etc.
- Modify Element: Change content, style, attributes, add / remove etc.
- Page Updates: Changes are reflected on the web page

WEB DEVELOPMENT NOTES

– ASYNC JAVASCRIPT –

1. What is Asynchronous JS ?

Asynchronous JavaScript allows us to run tasks in the background without blocking the execution of other code. It helps in handling operations like API calls, timers, file reading, etc. efficiently.

2. Why Async ?

- Prevents blocking the main thread
- Improves performance
- Better user experience

3. setTimeout()

Executes a function after a specified delay.

```
setTimeout(() => {
   console.log("Hello after 2 sec");
}, 2000);   // delay in ms
```

4. Callback Function

A function passed as an argument to another function.

```
function greet(name, callback) {
  console.log("Hi " + name);
  callback();
}

greet("Aman", () => {
  console.log("Callback Called");
});
```

5. Promises

A Promise represents a value that may be available now, or in the future, or never.

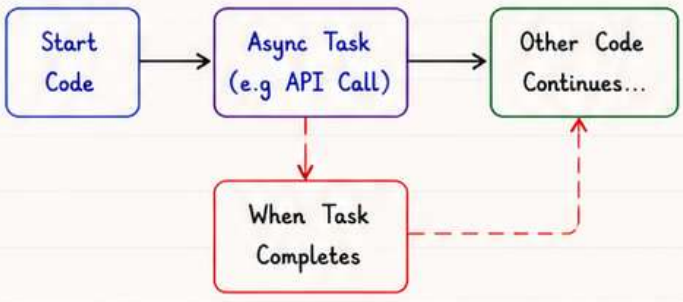
```
let promise = new Promise((resolve, reject) => {
  let success = true;
  if (success) resolve("Data Loaded");
  else reject("Something went wrong");
});

promise.then((res) => console.log(res))
    .catch((err) => console.log(err));
```

6. async / await

A modern way to write asynchronous code that looks synchronous.

```
async function fetchData() {
  try {
    let res = await fetch("api/data");
    let data = await res.json();
    console.log(data);
  } catch (err) {
    console.log("Error:", err);
  }
}
```

7. Async Flow



WEB DEVELOPMENT NOTES

— REACT JS (BASICS) —

1. What is React ?

React is a JavaScript library for building user interfaces, especially single page applications.
It is component based, fast and maintainable.

2. Features

- Component Based
- Reusable UI Components
- Virtual DOM (Better Performance)
- One-way Data Binding
- Declarative and Easy to Learn

3. Create React App

```
npx create-react-app my-app
# creates a new React project
cd my-app      # go to project folder
npm start      # start development server
```

4. JSX (JavaScript XML)

JSX lets us write HTML like syntax in JavaScript.

```
const element = (
  <h1 className="title">
    Hello, React!
  </h1>
);
```

Note : In JSX, use className instead of class and always return a single parent element.

5. Components

React is all about components.
A component is a reusable piece of UI.

Functional Component Example :

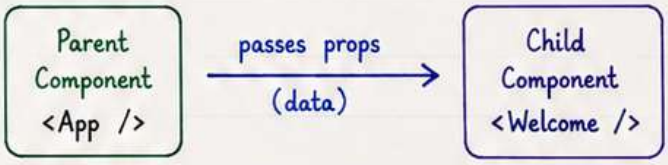
```
function Welcome(props) {
  return (
    <h2>Hello, {props.name} 👋</h2>
  );
}

export default Welcome;
```

6. Using Component

```
import Welcome from './Welcome';

function App() {
  return (
    <div>
      <Welcome name="Aman" />
    </div>
  );
}
export default App;
```

7. Props

Props (properties) are used to pass data from one component to another.



Props are read-only.
Data flows in one direction (Parent → Child).

WEB DEVELOPMENT NOTES

– REACT JS (BASICS) –

1. JSX (JavaScript XML)

JSX lets us write HTML like syntax in JavaScript.

```
const element = (
  <h1 className="title">
    Hello, React!
  </h1>
);
```

Note : In JSX, use className instead of class.

2. Rendering an Element

React renders elements to the DOM.

```
import { createRoot } from 'react-dom/client';

const root = createRoot(document.
      getElementById("root"));

root.render(<h1>Hello, React!</h1>);
```

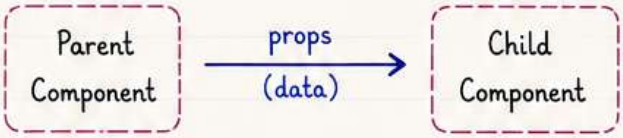
3. Components

Components are reusable pieces of UI.

```
function Welcome() {
  return (
    <h2>Welcome to React!</h2>
  );
}

export default Welcome;
```

4. Props

Props (properties) are used to pass data from one component to another.




```
function Greet(props) {
  return <p>Hello, {props.name}!</p>;
}

Greet.defaultProps = {
  name: "Guest"
};
```

5. State

State is used to manage dynamic data in a component.

```
import { useState } from 'react';
function Counter() {
  const [count, setCount] = useState(0);
  return (
    <button onClick={() => setCount(count + 1)}>
      Count: {count}
    </button>
  );
}
```

6. Event Handling

Events in React are written in camelCase.

```
function Button() {
  function handleClick() {
    alert("Button Clicked!");
  }
  return <button onClick={handleClick}>Click Me</button>;
}
```

Tip : React is all about components, state and props.
Build small components and reuse them. ☺

WEB DEVELOPMENT NOTES

— REACT JS (BASICS) —

7. useEffect Hook

useEffect lets you perform side effects in functional components.

```
import { useEffect } from 'react';
useEffect(() => {
    console.log("Component Mounted");
}, []);    // empty array = run once
```

8. Conditional Rendering

UI can be rendered conditionally based on a condition.

```
function User({ isLoggedIn }) {
  return (
    <div>
      {isLoggedIn ? <p>Welcome!</p> :
        <p>Please Login</p>}
    </div>
  );
}
```

9. Lists & Keys

Use map() to render lists. Always add a unique key.

```
const items = ["Apple", "Banana", "Mango"];
return (
 <ul>
   {items.map((item, index) => (
     <li key={index}>{item}</li>
   ))}
 </ul>
);
```

10. Handling Events

React uses camelCase for events.
Example: onClick, onChange

```
function Button() {
  const handleClick = () => {
    alert("Button Clicked!");
  };
  return <button onClick={handleClick}>
    Click Me
  </button>;
}
```

11. Passing Data with Props

Props help in passing data from parent to child component.

```
// Parent Component
<User name="Aman" age={25} />
- - - - - - - - - - - - - - - - - - - -
// Child Component
function User(props) {
  return <p>{props.name} is {props.age} years old.</p>
}
```

12. Default Props

You can set default values for props.

```
function User({ name = "Guest" }) {
  return <p>Hello, {name}!</p>;
}
```

Tip : Start small. Break UI into components and build step by step.
Practice by building mini projects! ☺

WEB DEVELOPMENT NOTES

— REACT JS (BASICS) —

13. Authentication in React

- What is Authentication ?

Authentication is the process of verifying the identity of a user before allowing access.

Common Methods

- Username / Password
- Token Based (JWT)
- OAuth (Google, GitHub, etc.)

14. Token Based Auth (JWT)

After login, server sends a token which is used for future requests.

Flow :

1. User Login
↓
2. Server validates & returns JWT
↓
3. Store token (localStorage / cookie)
↓
4. Send token in request headers
↓
5. Server validates token

15. Store Token

We can store token in localStorage or cookies.

```
// Save Token
localStorage.setItem("token", token);

// Get Token
const token = localStorage.getItem("token");
```

16. Protected Route

We can restrict pages which should be accessed only after login.

```
import { Navigate } from 'react-router-dom';

function ProtectedRoute({ children }) {
  const token = localStorage.getItem("token");
  if (!token) {
    return <Navigate to="/login" />;
  }
  return children;
}
```

17. Login Example

```
function handleLogin(e) {
  e.preventDefault();
  // call API
  login(email, password).then(res => {
    localStorage.setItem("token", res.token);
    navigate("/dashboard");
  }).catch(err => {
    setError("Invalid credentials");
  });
}
```

18. Logout

```
function handleLogout() {
  localStorage.removeItem("token");
  navigate("/login");
}
```

Note :
- Never store sensitive data in localStorage.
- Use HTTPS for secure communication.
- JWT token has an expiry (exp) time.

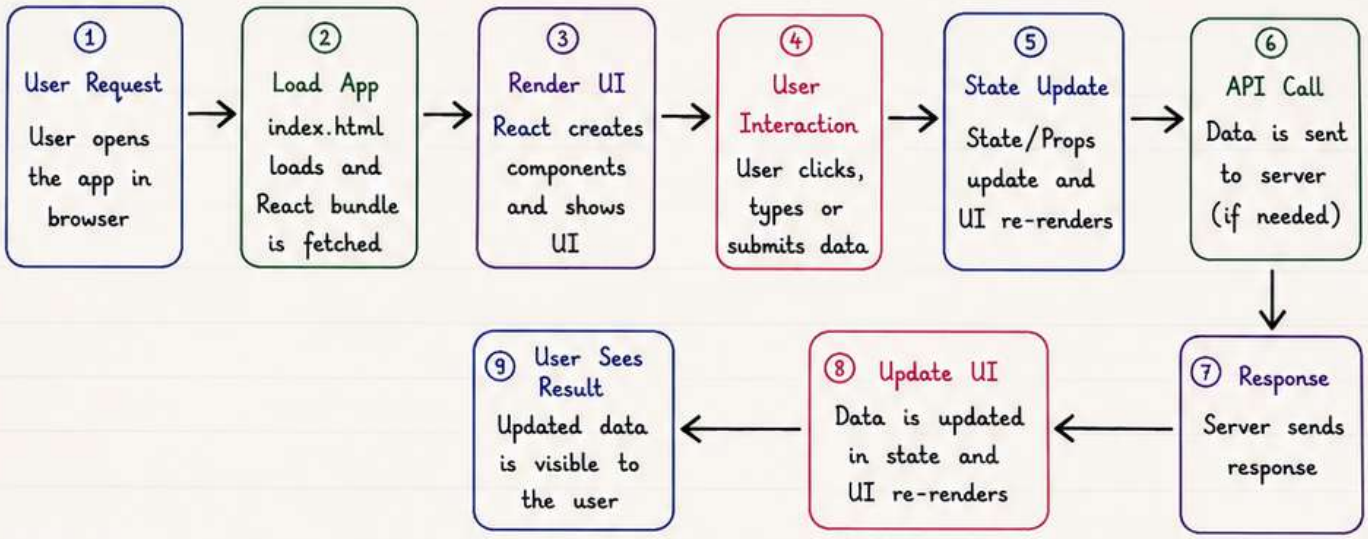
Tip : Authentication keeps your app secure. Always protect important routes and handle tokens safely! ☺

WEB DEVELOPMENT NOTES

— REACT JS (BASICS) —

19. Complete Web Application Flow (React)

High Level Flow :



Step by Step Details :

1. **User Request**
   - User enters URL or opens the app.
2. **Load App**
   - index.html loads.
   - React JS file (bundle) is downloaded.
3. **Render UI**
   - React builds the component tree.
   - Initial UI is displayed on screen.
4. **User Interaction**
   - User performs actions (click, type, submit).
5. **State Update**
   - State or props change.
   - React re-renders only changed parts.
6. **API Call**
   - App makes API call using fetch/axios.
7. **Response**
   - Server processes request and sends data.
8. **Update UI**
   - Data is stored in state.
   - UI updates automatically.
9. **User Sees Result**
   - User sees the updated information.

React follows a one-way data flow which makes the app predictable and easy to debug.

Key Points :

- UI is built using reusable components.
- State drives the UI.
- Data flows from parent to child via props.
- Side effects (like API calls) can be handled using useEffect.
- Always keep components small and focused.

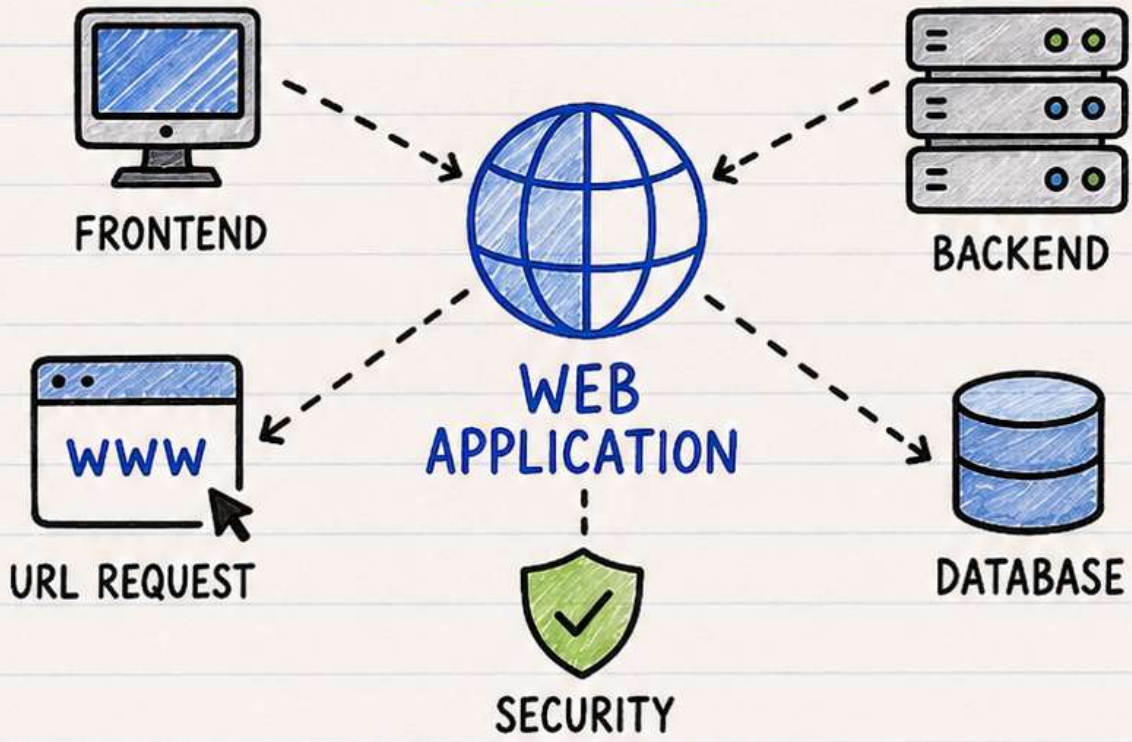
Example Flow (Login Page) :

User opens /login
↓
Login form is shown
↓
User enters email & password and clicks Login
↓
API call is made to /api/login
↓
Server validates and sends response
↓
On success → token stored, user navigated to dashboard

Remember :
Build small. Test often. Keep it simple. That's the React way! ☺

COMMENT WEB DEVELOPMENT NOTES

FOR PDF NOTES



LIKE | SHARE | FOLLOW

COMMENT "PDF" FOR WEB DEVELOPMENT NOTES